

Table des matières

1. Notes paiement et prix (declinaisons)
 - 1) Calcul du prix avec declinaisons et TVA
 - 2) Eviter l'erreur de paiement (PS_OS_ERROR)
 - 3) Selection du mode de paiement et cle PS_OS_*
 - 4) Resolution du statut de commande
 - 5) Transporteur apres creation
 - 6) Endpoints Webservice utilises

Notes paiement et prix (declinaisons)

Ce document resume ce qui a ete fait pour:

- Calculer les prix corrects (declinaisons, prix specifiques, taxes)
- Construire la commande sans erreur de paiement
- Lier le mode de paiement a la cle de configuration (PS_OS_*)

1) Calcul du prix avec declinaisons et TVA

Dans `new_app/src/services/CommandeService.js`:

- Recuperation des produits via `productService.getOne(productId)`
- Recuperation du taux de taxe via `productService.getTaxRateByGroupId(taxGroupId)`
- Recuperation des declinaisons via `productService.getCombinationsByProductId(productId)`
- Recuperation des prix specifiques via `productService.getSpecificPrices(productId)`

Logique:

- Prix HT de base = `product.price`
- Impact declinaison = `combo.priceImpact` (aussi appele `impactPrice` selon la source)
- Prix specifiques appliques via `productService.applySpecificPrice(...)`
- Prix TTC = $HT * (1 + taxRate / 100)$
- Totaux HT et TTC multiplies par `defaults.conversionRate`

Variables cle:

- `resolvedItemPriceHT(item)`
- `resolvedItemPriceTTC(item)`
- `total_products` = HT
- `total_products_wt` et `total_paid` = TTC

2) Eviter l'erreur de paiement (PS_OS_ERROR)

Probleme: PrestaShop recalcule `total_paid_tax_incl` (TTC). Si `total_paid_real` et `total_paid` ne sont pas coherents, la commande peut passer en etat erreur (PS_OS_ERROR).

Solution appliquee:

- `total_paid` = TTC
- `total_paid_real` = 0.00 pour les paiements differe (COD, cheque, virement)

Cela evite la discordance et garde l'etat attendu.

3) Selection du mode de paiement et cle PS_OS_*

Dans `new_app/src/services/CommandeService.js`:

- Liste `PAYMENT_CONFIG_KEYS` avec `configKey`, `module`, `label`
- `getPaielements()` lit `/configurations?display=[name,value]&filter[name]=[...]`
- On ne garde que les cle actives (`value != 0`)

Dans `new_app/src/views/ShopCartView.vue`:

- La liste deroulante affiche maintenant la cle:
 - `{{ p.label }}` - `{{ p.configKey || 'cle manquante' }}`
- La valeur selectionnee est `configKey`
- Fallback clair si `configKey` manquante:
 - Message: "Cle de configuration manquante: fallback sur PS_OS_COD_VALIDATION."
 - `configKey` forcee a `PS_OS_COD_VALIDATION`

4) Resolution du statut de commande

Dans `new_app/src/services/CommandeService.js`:

- `getOrderStateIdByConfigKey(configKey)` interroge `/configurations` et recupere l'id
- Si echec: fallback par libelle (`getEtat(langId)`)
- `payload.current_state` est force si un id valide est trouve
- Apres creation, `setOrderState(orderId, orderStateId)` force l'etat voulu

5) Transporteur apres creation

Dans `new_app/src/services/CommandeService.js`:

- `updateOrderCarrier(orderId, carrierId)` met a jour `id_carrier` dans `orders`
- Puis met a jour ou cree `order_carriers` avec frais a 0

Dans `new_app/src/views/ShopCartView.vue`:

- Liste deroulante des transporteurs chargee apres la creation
- `assignCarrierAndSetCodState(orderId, carrierId)` met a jour le transporteur puis force l'etat "payment after delivery" si disponible

6) Endpoints Webservice utilises

- GET `/configurations?display=[name,value]&filter[name]=[...]`
- GET `/order_states?display=[id,name,logable,deleted]`
- GET `/orders?display=[...]&filter[...]`
- POST `/orders`

- POST /order_histories
- GET /order_carriers?filter[id_order]=[...]&display=full
- PUT /order_carriers/{id} ou POST /order_carriers
- GET /orders/{id} et PUT /orders/{id}